Algoritma & Struktur Data

Week 3 - Linked List - 1

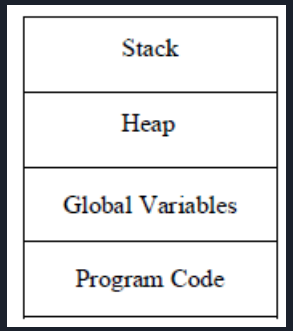
Last Week Review

- Data structure
  - User defined data type and a composite data type consist of one or more value with different data type.
- Enumeration
  - User defined data type to provide readability to a constant which will have it's value defined automatically if it's not defined.
  - Resume the sequence of last previous defined enum.
- Union
  - User defined data type which share same memory address, the address size decided by the largest data type in that union.
- File processing
  - How to read file
  - How to write file
  - Several option to access the file
    - a,r,w
    - a+,r+,w+

Outline

- Heap vs Non Heap Memory
  - Concept
  - Implementation
- Single Linked List
  - Definition
- Basic Single Linked List operation
  - Initialization
  - Add
  - Print
  - Delete List
- Circular Single Linked List
  - Definition
  - Implementation

Dynamic Memory Allocation



Four regions of memory

- We can write programs and obtain memory as they are running
- With dynamic memory allocation, memory is not reserved or set aside at the start of the program; rather it is allocated on an as-needed basis

Heap Vs Non heap Memory (Concept)

Non Heap	Heap
Both Use Ram	
Allocated During function call only	Allocated on programmer demand
Automatically Allocated	Manually Allocated
Automatically Deallocated	Manually Deallocated

Dynamic Memory Allocation

Functions	Description
malloc()	The malloc() function allocates a block of size bytes from the memory heap.
calloc	calloc() allocates a block of size bytes from the memory heap and block is initialized to zero.
realloc()	realloc() function reallocates the memory block that means it attempts to reduce or increase the previously allocated block.
free()	free() function is used to de-allocate a memory block allocated by a previous call to calloc(), malloc(), or realloc().

Implementation

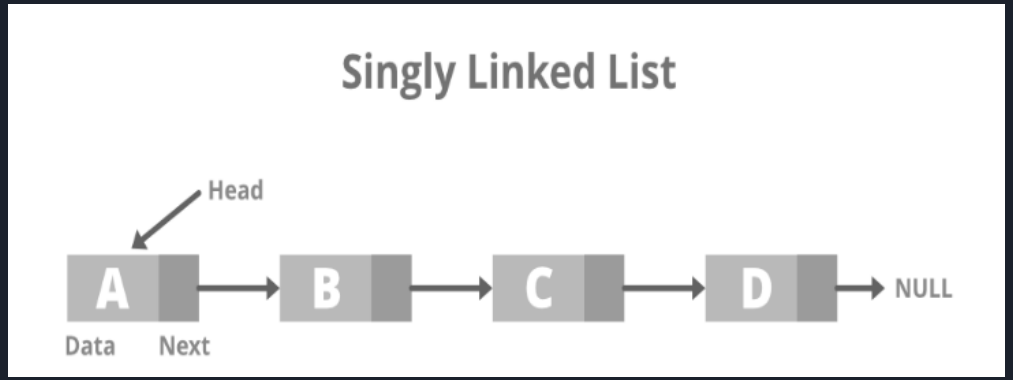
```c
#include<stdio.h>
#include<malloc.h>
int main()
{
    //allocate all memory required in function
    //in this case, single integer
    int a[10];
    a[0] = 0;
    printf("%d",a[0]);
    return 0;
    //deallocate all memory required
    //in function in this case, single integer
}
```

```c
#include<stdio.h>
#include<malloc.h>
int main()
{
    //allocate all memory required in function
    //in this case, single integer
    int *a;
    a = (int *) malloc(sizeof(int)*10);
    a[0] = 0;
    printf("%d",a[0]);
    return 0;
    //deallocate all memory required
    //in function in this case, single integer
}
```

int *var;
int **var;
Format: var = (tipe_data *) malloc(sizeof(tipe_data)*size)
Format: var2 = (tipe_data **) malloc(sizeof(tipe_data*)*size)

Single Linked List (Definition)

- A list of struct which include one or more data
- Every struct need to have a pointer
- The simplest form of single linked list
- The pointer will be the connection between data
- Imagine a train with one way door



Reference: https://www.geeksforgeeks.org/types-of-linked-list/

Advantages of Linked List

- Linked list are **dynamic data structures**. That is, they can extend or shrink during the execution of a program.
- Storage need not be contiguous.
- Efficient memory utilization. Here memory is not pre-allocated. Memory is allocated whenever it is required.
- Insertion or deletion is easy and efficient, may be done very fast, in a constant amount of times, independent of the size of the list.
- The joining of two linked lists can be done by assigning pointer of the second linked list in the last node of the first linked list.

Basic Single Linked List operation (Initialization)

- During initialization we usually do these step
  - Create the Data Structure
  - Head → Pointing to the front of the list
  - Curr → Pointing to the current node
  - Node → Newly created data
  - Tail → Pointing to the end of the list

- Step of Initialization:
  - Set the Head, Curr and Tail NULL (empty state)
  - Node can be nulled or not for initialization.

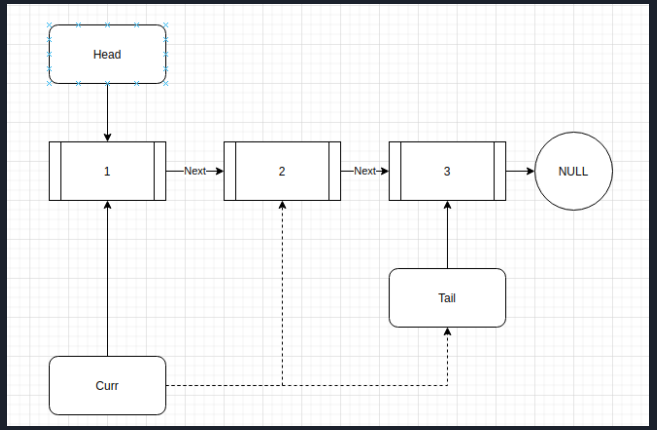
```c
#include<stdio.h>
#include<malloc.h>

typedef struct element{
    int value;
    struct element *next;
};

int main()
{
    struct element *head;
    struct element *tail;
    struct element *curr;
    struct element *node;
    head = tail = curr = NULL;
}
```

Basic Single Linked List operation (Add)

- During Add Operation we usually conduct these step:
  - Allocate memory for node and fill the value.
  - Set the next pointer of node to NULL
  - Check if the list is empty (check head)
  - If it's empty:
    - Set head, tail and curr to the new node
  - If it's not empty
    - Set next pointer of tail to new node
    - Set tail pointer to it's next pointer



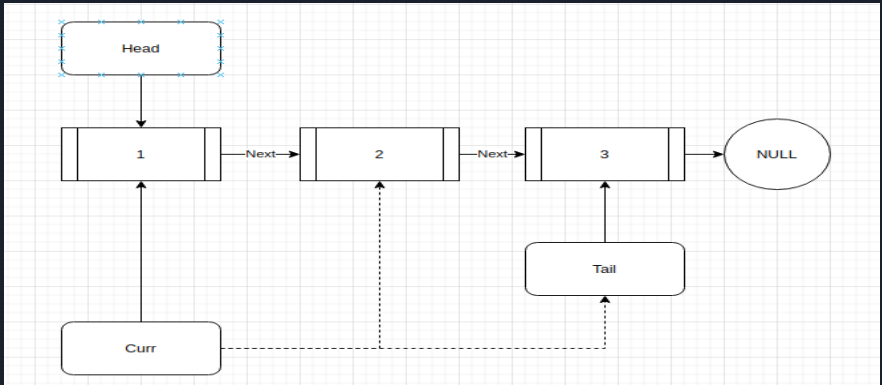

```c
#include<stdio.h>
#include<malloc.h>

typedef struct element{
    int value;
    struct element *next;
};

int main()
{
    struct element *head;
    struct element *tail;
    struct element *curr;
    struct element *node;
    head = tail = curr = NULL;
    int value;
    while(scanf("%d",&value) == 1){
        node = (struct element *) malloc(sizeof(struct element));
        node->value = value;
        node->next = NULL;
        if(head == NULL){
            head = tail = node;
        }
        else {
            tail->next = node;
            tail = tail->next;
        }
    }
}
```

Basic Single Linked List operation (Print)

- During Print Operation we usually conduct these step:
  - Set Curr pointer to head.
  - As long as curr pointer is not null
    - Print value
    - Set curr pointer to its next pointer



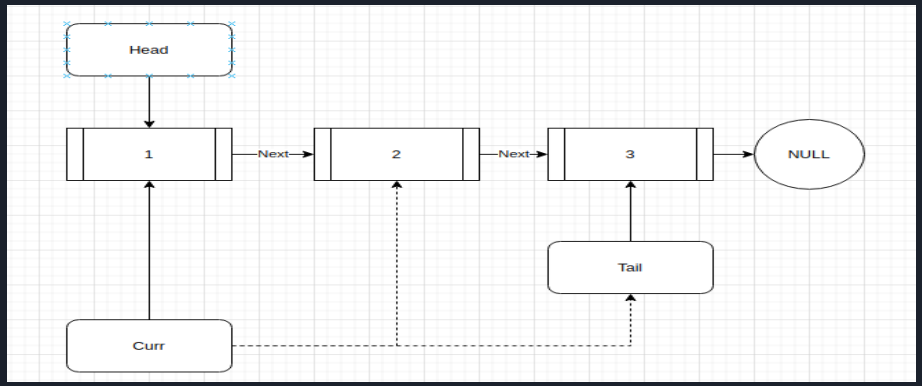

```c
#include<stdio.h>
#include<malloc.h>

typedef struct element{
    int value;
    struct element *next;
};

int main()
{
    struct element *head;
    struct element *tail;
    struct element *curr;
    struct element *node;
    head = tail = curr = NULL;
    int value;
    while(scanf("%d",&value) == 1){
        node = (struct element *) malloc(sizeof(struct element));
        node->value = value;
        node->next = NULL;
        if(head == NULL){
            head = tail = node;
        }
        else {
            tail->next = node;
            tail = tail->next;
        }
    }
    curr = head;
    while(curr){
        printf("%d ",curr->value);
        curr = curr->next;
    }
}
```

Basic Single Linked List operation (Delete Whole List)

- During Print Operation we usually conduct these step:
  - Set Curr pointer to head.
  - As long as curr pointer is not null
    - Set head to it's head next pointer
    - Free curr
    - Set curr = head



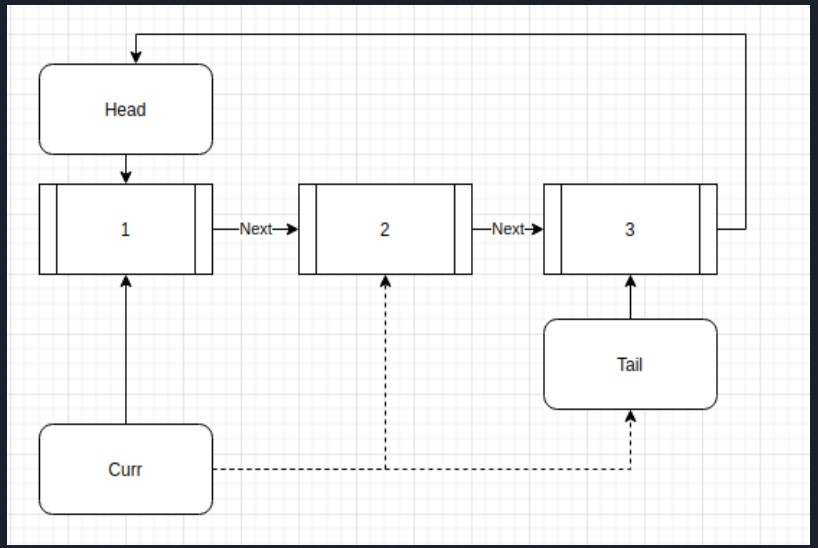

```c
#include<stdio.h>
#include<malloc.h>

typedef struct element{
    int value;
    struct element *next;
};

int main()
{
    struct element *head;
    struct element *tail;
    struct element *curr;
    struct element *node;
    head = tail = curr = NULL;
    int value;
    while(scanf("%d",&value) == 1){
        node = (struct element *) malloc(sizeof(struct element));
        node->value = value;
        node->next = NULL;
        if(head == NULL){
            head = tail = node;
        }
        else {
            tail->next = node;
            tail = tail->next;
        }
    }
    curr = head;
    while(curr){
        printf("%d ",curr->value);
        curr = curr->next;
    }
    curr = head;
    while(curr){
        head = head->next;
        free(curr);
        curr = head;
    }
}
```

Circular Linked List (Definition & Implementation)

- Definition
  - Circular Linked list is a linked list where the tail never point to null, but always to its head.
- Real Life Implementation
  - It can be used to make a round turn based game (circular turn game such as Monopoly, UNO)
  - OS scheduler to multiple process.
- Class Discussion:
  - Can you try to change the code in previous slide to circular linked list?



Exercises

1. Write an algorithm to insert a data X after a specific data item Y in the linked list.
2. Write an algorithm to delete all nodes having greater than a given value, from a given singly linked list.

THE END